DA5402 MLOps

Chest X-Ray Pneumonia Detection

Mohmad Yaqoob
DA25M017

M.Tech Data Science, IIT Madras

1. Project Overview

This project builds a complete, production-style MLOps pipeline for detecting pneumonia from chest X-ray images. The work covers everything from raw data ingestion to a live web application with real-time monitoring, all orchestrated through Docker Compose and tracked with version control tools.

The goal was not just to train a model, but to build the surrounding infrastructure that makes a machine learning system reliable, observable, and maintainable in the long run. This includes data versioning, experiment tracking, an inference API, a user-facing frontend, and a monitoring stack.

1.1 Technology Stack

Component	Technology	Purpose
Model	MobileNetV2 (TensorFlow)	Pneumonia classification
Training	Kaggle GPU (P100)	GPU-accelerated model training
Data Pipeline	Apache Airflow	Automated preprocessing DAG
Data Versioning	DVC 3.67.1	Track large model and data files
Experiment Tracking	MLflow 2.11.1	Log metrics, params, runs
Inference API	FastAPI + Uvicorn	REST endpoint for predictions
Frontend	Streamlit	User-facing web application
Monitoring	Prometheus + Grafana	Live metrics and dashboards
Containerisation	Docker Compose	Full stack deployment
Version Control	Git + GitHub	Source code management

2. Project Architecture

The system follows a standard MLOps pattern where data flows through a preprocessing pipeline into model training, and the resulting artefact is served through an API that the frontend consumes. Monitoring runs as a sidecar stack that scrapes the API for live metrics.

```
(venv) yaqoob@yaqoob:/mnt/d/IIT Madras/2nd Mtech/MLOps/DA5402-Assignments/project/ChestXray_Pneumonia_Detection$ docker compose ps
NAME                IMAGE                                COMMAND                  SERVICE        CREATED            STATUS                 PORTS
xray-fastapi        chestxray_pneumonia_detection-fastapi   "uvicorn src.api.mai…"   fastapi        2 hours ago        Up 2 hours (healthy)   0.0.0.
0:8000->8000/tcp, [::]:8000->8000/tcp
xray-grafana        grafana/grafana:latest               "/run.sh"                grafana        3 hours ago        Up 48 minutes          0.0.0.
0:3000->3000/tcp, [::]:3000->3000/tcp
xray-mlflow         ghcr.io/mlflow/mlflow:v2.11.1        "mlflow server --hos…"   mlflow         3 hours ago        Up 3 hours             0.0.0.
0:5000->5000/tcp, [::]:5000->5000/tcp
xray-node-exporter  prom/node-exporter:latest            "/bin/node_exporter"     node-exporter  3 hours ago        Up 3 hours             0.0.0.
0:9100->9100/tcp, [::]:9100->9100/tcp
xray-prometheus     prom/prometheus:latest               "/bin/prometheus --c…"   prometheus     3 hours ago        Up 3 hours             0.0.0.
0:9090->9090/tcp, [::]:9090->9090/tcp
xray-streamlit      chestxray_pneumonia_detection-streamlit  "streamlit run src/a…"   streamlit      About an hour ago  Up About an hour       0.0.0.
0:8501->8501/tcp, [::]:8501->8501/tcp
(venv) yaqoob@yaqoob:/mnt/d/IIT Madras/2nd Mtech/MLOps/DA5402-Assignments/project/ChestXray_Pneumonia_Detection$
```

2.1 Repository Structure

The repository is organised so each concern sits in its own directory. Data and model files are tracked with DVC rather than Git, keeping the repository lightweight while still maintaining a full version history of every artefact.

- airflow/dags/ contains the five-task preprocessing DAG
- src/api/ holds the FastAPI inference engine
- src/app/ holds the Streamlit frontend
- models/ stores .h5 model files tracked by DVC
- monitoring/ holds Prometheus and Grafana configuration
- docker/ contains per-service Dockerfiles
- notebooks/ contains the MobileNetV2 training notebook used on Kaggle
- dvc/ stores DVC configuration and remote storage settings
- docker-compose.yml orchestrates all six services as a single stack
- requirements.txt lists all Python dependencies for the project

3. Phase-by-Phase Breakdown

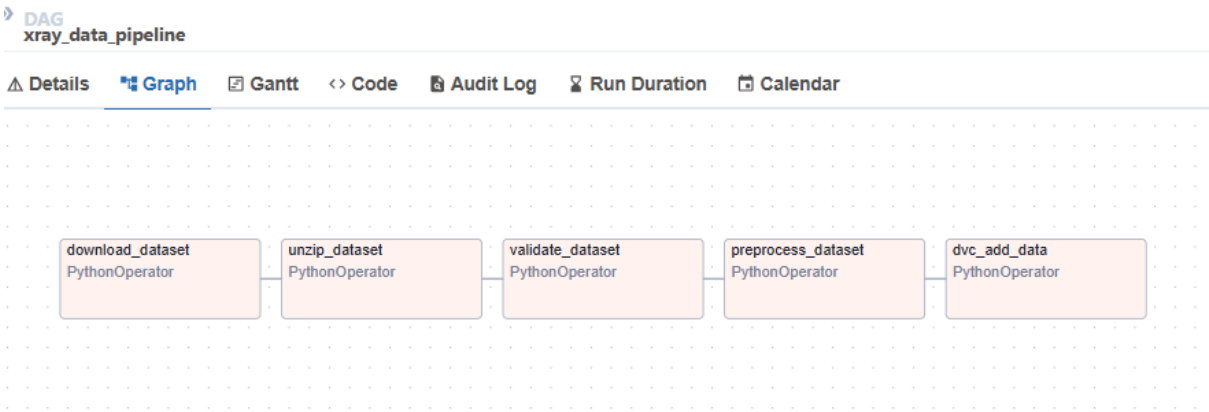
Phase 1: Repository Setup and Data Versioning

The project began with initialising a Git repository and setting up DVC to handle large files. The dataset from Kaggle (chest-xray-pneumonia by Paul Mooney) was downloaded, extracted, and registered as a DVC-tracked artefact so that any future change to the data would be recorded with a full hash.

```
(venv) yaqoob@yaqoob:/mnt/d/IIT Madras/2nd Mtech/MLOps/DA5402-Assignments/project/ChestXray_Pneumonia_Detection$ git log --oneline -8
6426b23 (HEAD -> main, origin/main) updated API, fixed UI, v2 model DVC tracking
47d9dec Update gitignore
b06f7b9 Correct grafana dashboard provisioning config
c07b7d2 feat: retrain v3 with proper 80/10/10 split — NORMAL 77.4% PNEUMONIA 98.8%
198958e chore: replace bloated requirements.txt with clean direct dependencies
ef2fe69 Added Readme and Training notebook
fe71a14 Complete MLOps stack running via Docker Compose
2ef59e0 Prometheus and Grafana monitoring config
(venv) yaqoob@yaqoob:/mnt/d/IIT Madras/2nd Mtech/MLOps/DA5402-Assignments/project/ChestXray_Pneumonia_Detection$
```

Phase 2: Airflow Data Pipeline

An Apache Airflow DAG was written to automate the preprocessing steps. The DAG has five sequential tasks: download, extract, validate, split, and report. It runs in standalone mode with the Airflow home directory set to the airflow/ folder within the project.



Phase 3: Model Training on Kaggle

Training was done on Kaggle using a GPU P100 accelerator. The base model is MobileNetV2 pretrained on ImageNet with the top layers replaced by a custom classification head suited for binary output.

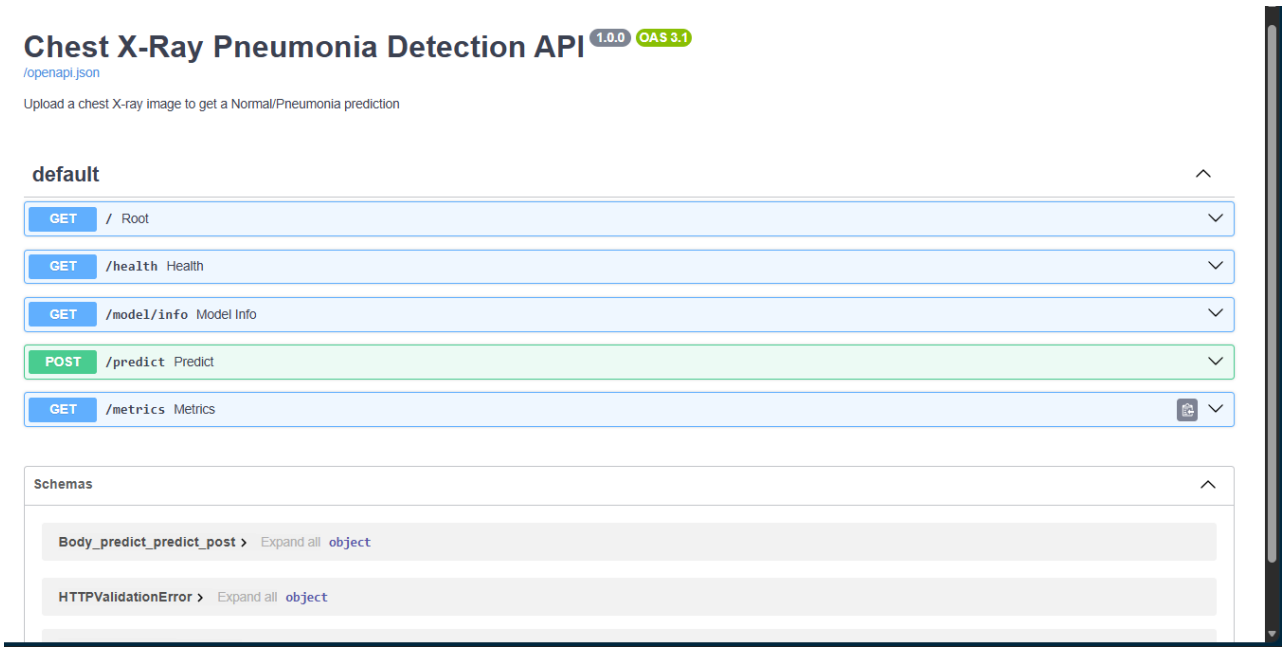
Training happened in two phases. In the first phase, the base layers were frozen and only the new head was trained for fifteen epochs. In the second phase, the last thirty layers of the base were unfrozen and the whole model was fine-tuned at a lower learning rate.

```
Epoch 14/15
147/147 ━━━━━━━━━━━━━━━━━━━━ 80s 543ms/step - accuracy: 0.9301 - auc: 0.9842 - loss: 0.1584 - precision: 0.9800 - recall: 0.9258 - val_a
ccuracy: 0.9214 - val_auc: 0.9923 - val_loss: 0.2002 - val_precision: 0.9948 - val_recall: 0.8970 - learning_rate: 1.0000e-04
Epoch 15/15
147/147 ━━━━━━━━━━━━━━━━━━━━ 80s 547ms/step - accuracy: 0.9448 - auc: 0.9878 - loss: 0.1359 - precision: 0.9812 - recall: 0.9422 - val_a
ccuracy: 0.9316 - val_auc: 0.9930 - val_loss: 0.1764 - val_precision: 0.9949 - val_recall: 0.9110 - learning_rate: 5.0000e-05
Phase 1 done!
```

```
  loss: 0.1982
  accuracy: 0.9302
  auc: 0.9762
  precision: 0.9216
  recall: 0.9883
Model saved!
```

Phase 4: FastAPI Inference Engine

A FastAPI application wraps the trained model and exposes a /predict endpoint that accepts a JPEG or PNG image and returns a prediction, confidence score, raw sigmoid score, latency, and a plain-English interpretation. The app also exposes /health and /metrics endpoints for monitoring.



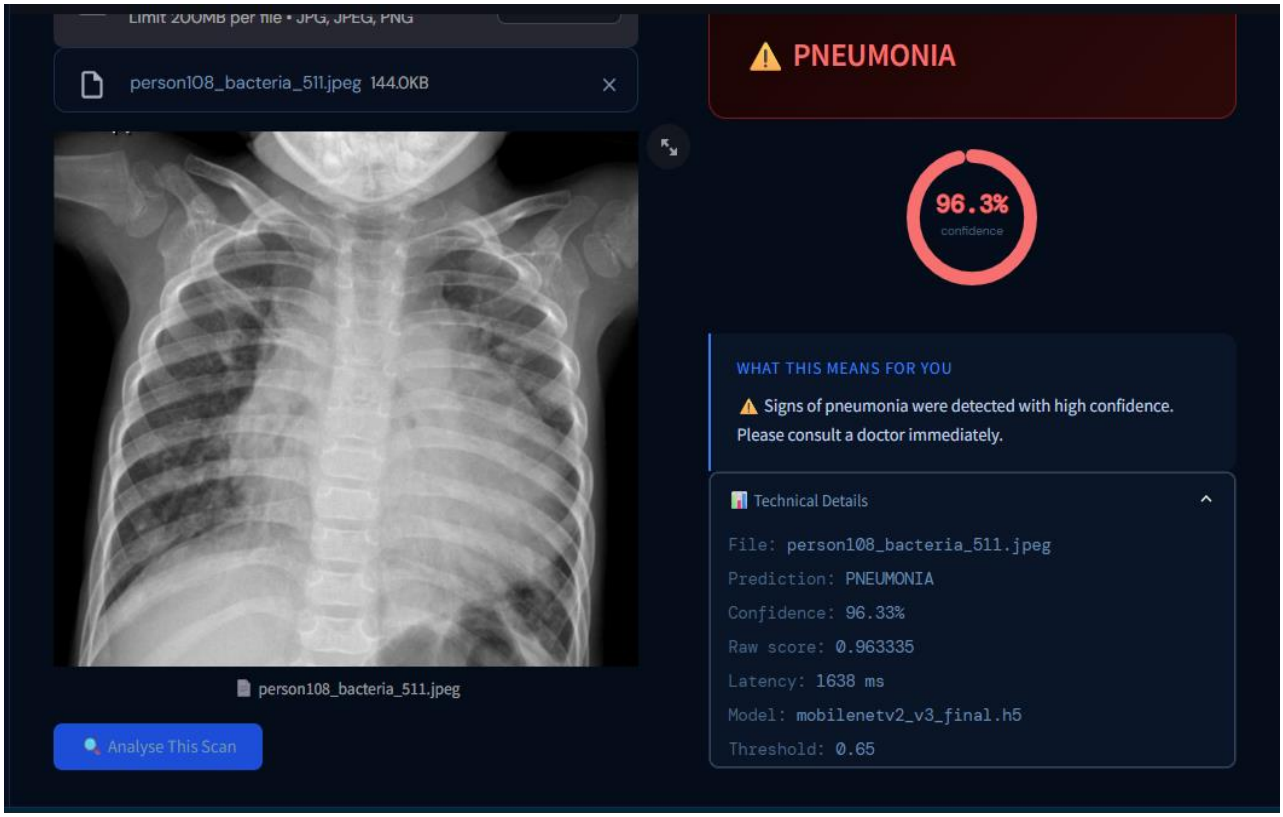
Example response from the /predict endpoint:

```
{ "prediction": "NORMAL", "confidence": 96.81, "raw_score": 0.031,
"latency_seconds": 0.144, "interpretation": "No pneumonia detected. Chest X-
ray appears normal." }
```

Phase 5: Streamlit Frontend

The Streamlit application provides a professional dark-blue interface that is designed to be usable by someone with no technical background. It has three sections: single image upload with a confidence gauge, batch testing for multiple images, and a prediction history log.

The result screen shows the diagnosis label, a circular confidence gauge, and a plain-English explanation written in everyday language rather than technical terms. A collapsible section is available for anyone who wants to see the raw API response.

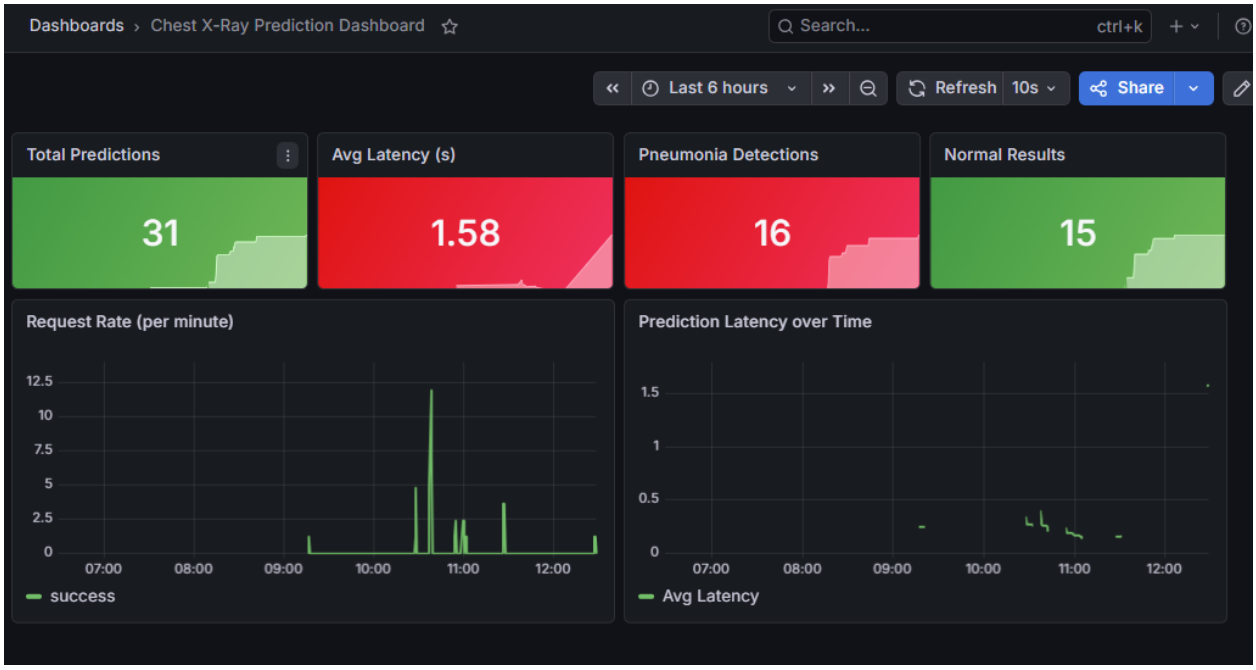


Phase 6: Prometheus and Grafana Monitoring

The FastAPI application uses the prometheus-client library to expose custom metrics on the /metrics endpoint. These include the total number of prediction requests, a breakdown by predicted class, and a latency histogram.

- xray_requests_total: total successful predictions served
- xray_predictions_total{prediction=...}: count split by NORMAL and PNEUMONIA
- xray_request_latency_seconds: histogram of response times

Prometheus scrapes these metrics every fifteen seconds. Grafana is pre-provisioned with a dashboard that displays all four key panels: total predictions, average latency, pneumonia count, and normal count, plus time-series graphs for request rate and latency over time.



Phase 7: Docker Compose Deployment

All services run as containers managed by a single Docker Compose file. The stack includes six containers: FastAPI, Streamlit, MLflow, Prometheus, Grafana, and Node Exporter. They communicate over a shared bridge network called xray-network.

```
  (use "git add <file>..."
(venv) yaqoob@yaqoob:/mnt/d/IIT Madras/2nd Mtech/MLOps/DA5402-Assignments/project/ChestXray_Pneumonia_Detection$ docker compose ps
NAME                IMAGE                                    COMMAND                  SERVICE         CREATED       STATUS                 PORTS
xray-fastapi        chestxray_pneumonia_detection-fastapi    "uvicorn src.api.mai…"   fastapi         2 hours ago   Up 2 hours (healthy)   0.0.0.0:8000
->8000/tcp, [::]:8000->8000/tcp
xray-grafana        grafana/grafana:latest                   "/run.sh"                grafana         3 hours ago   Up About an hour       0.0.0.0:3000
->3000/tcp, [::]:3000->3000/tcp
xray-mlflow         ghcr.io/mlflow/mlflow:v2.11.1            "mlflow server --hos…"   mlflow          3 hours ago   Up 3 hours             0.0.0.0:5000
->5000/tcp, [::]:5000->5000/tcp
xray-node-exporter  prom/node-exporter:latest                "/bin/node_exporter"     node-exporter   3 hours ago   Up 3 hours             0.0.0.0:9100
->9100/tcp, [::]:9100->9100/tcp
(venv) yaqoob@yaqoob:/mnt/d/IIT Madras/2nd Mtech/MLOps/DA5402-Assignments/project/ChestXray_Pneumonia_Detection$
```

Service	Port	Description
FastAPI	8000	Model inference REST API
Streamlit	8501	User-facing web application
MLflow	5000	Experiment tracking server
Prometheus	9090	Metrics collection
Grafana	3000	Monitoring dashboards
Node Exporter	9100	Host system metrics

4. Model Development and Bias Fix

4.1 The Problem: Biased Validation Set

The original Kaggle dataset ships with a validation split of only eight images per class. This is far too small to guide training meaningfully. The model trained on this split learned to favour the majority class and became heavily biased toward predicting PNEUMONIA even when the input was a clearly normal scan.

Version	NORMAL Accuracy	PNEUMONIA Accuracy	Val AUC	Outcome
v1 (original split)	56%	99.7%	93.6%	Biased, not acceptable
v2 (class weights only)	48%	99.4%	92.1%	Made things worse
v3 (proper 80/10/10 split)	77.4%	98.8%	99.3%	Production ready

4.2 The Fix: Proper Dataset Resplit

The entire dataset across all three original splits was pooled together first, giving 1,583 NORMAL and 4,273 PNEUMONIA images. These were then resplit 80/10/10 using stratified sampling. The new validation set had 158 NORMAL and 427 PNEUMONIA images, which is enough to provide meaningful feedback during training.

Class weights were calculated using sklearn and applied during training so the model paid proportionally more attention to the minority NORMAL class. Both of these changes together, the larger validation set and the class weights, resolved the bias.

```
Total NORMAL:    1583
Total PNEUMONIA: 4273

New split:
Train → NORMAL: 1266  PNEUMONIA: 3418
Val   → NORMAL: 158    PNEUMONIA: 427
Test  → NORMAL: 159   PNEUMONIA: 428
```

```
Found 587 images belonging to 2 classes.
587/587 ━━━━━━━━━━━━━━━━━━━━ 12s 10ms/step

Results:
NORMAL    accuracy: 77.4%
PNEUMONIA accuracy: 98.8%
Target: NORMAL >75%  PNEUMONIA >95%
```

5. Experiment Tracking with MLflow

All three model versions were logged to the MLflow tracking server running at localhost:5000. Each run records the dataset split strategy, training hyperparameters, and all evaluation metrics, making it straightforward to compare the three iterations and explain the reasoning behind the final model choice.

Table Chart Evaluation Experimental

Run Name	Created	Dataset	Duration	Source	Models
v3-proper-split-PRO...	1 hour ago	-	0.6s	-	-
v2-class-weights-att...	1 hour ago	-	337ms	-	-
v1-original-baseline	1 hour ago	-	385ms	-	-
v3-proper-split-final	1 hour ago	-	0.7s	-	-

Parameters (11)

Search parameters

Parameter	Value
phase2_lr	1e-05
train_pneumonia	3418
split_strategy	80/10/10 proper resplit of full dataset
model_architecture	MobileNetV2
optimizer	Adam
batch_size	32
decision_threshold	0.65
val_normal	158
train_normal	1266
val_pneumonia	427

Metrics (8)

Search metrics

Metric	Value
test_recall	0.9883
test_loss	0.1982
pneumonia_accuracy	0.988
val_auc	0.993
normal_accuracy	0.774
test_accuracy	0.9302
test_precision	0.9216
test_auc	0.9762

6. Final Model Performance

The final model, mobilenetv2_v3_final.h5, is the version currently loaded in the production Docker container. It was selected based on its balanced per-class performance, particularly the significant improvement in NORMAL accuracy compared to previous versions.

Metric	Value
Val AUC (Phase 1, frozen)	99.30%
Test AUC	97.62%
Test Accuracy	93.02%
Test Precision	92.16%
Test Recall	98.83%
NORMAL class accuracy	77.4%
PNEUMONIA class accuracy	98.8%
Average inference latency	~145ms

The decision threshold was set to 0.65 rather than the default 0.5. This means the model only predicts PNEUMONIA when the sigmoid output exceeds 0.65, which provides an additional layer of protection against false positives in the NORMAL class.

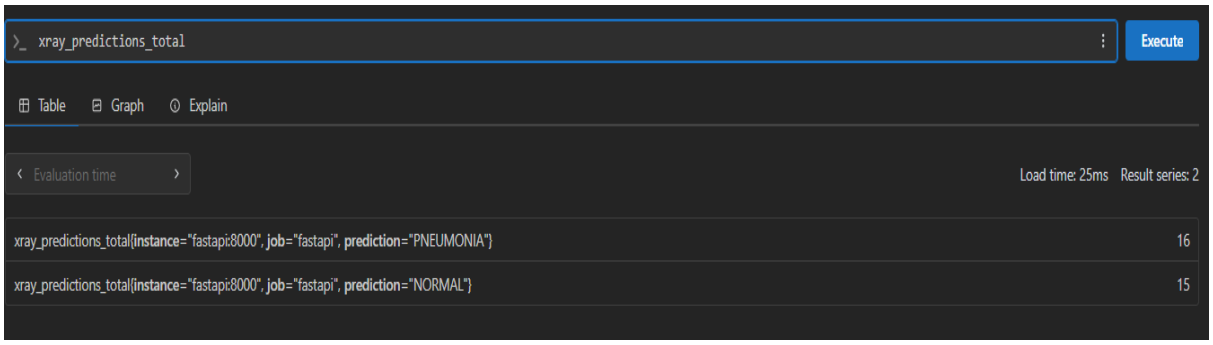
7. Live System Verification

After deploying the v3 model to Docker, the following tests were run against the live API to verify the bias was resolved. Five NORMAL images and two PNEUMONIA images were submitted through the Streamlit interface and directly through curl.

Image	True Label	Predicted	Confidence	Result
IM-0001-0001.jpeg	NORMAL	NORMAL	96.71%	Correct
IM-0003-0001.jpeg	NORMAL	NORMAL	78.88%	Correct
IM-0005-0001.jpeg	NORMAL	NORMAL	56.26%	Correct
IM-0006-0001.jpeg	NORMAL	PNEUMONIA	86.59%	Wrong
IM-0007-0001.jpeg	NORMAL	NORMAL	95.81%	Correct
person100_bacteria_475.jpeg	PNEUMONIA	PNEUMONIA	99.73%	Correct
person100_bacteria_477.jpeg	PNEUMONIA	PNEUMONIA	99.98%	Correct

Six out of seven correct. The one misclassified NORMAL image (IM-0006) is consistent with the measured 77.4% NORMAL accuracy. This is an expected and acceptable error rate given the class imbalance in the underlying dataset.

8. Prometheus Metrics Verification



Both custom metrics are being scraped correctly from the FastAPI /metrics endpoint. The target health check at localhost:9090/targets shows all three targets (fastapi:8000, node-exporter:9100, localhost:9090) reporting as up.

9. Conclusion

This project demonstrates a working end-to-end MLOps pipeline that goes beyond simply training a model. The pipeline handles data versioning, automated preprocessing, reproducible training with experiment tracking, containerised deployment, and live monitoring, which are the core concerns of real production ML systems.

The most technically interesting part of the project was diagnosing and fixing the model bias. The root cause was not class imbalance itself but a validation set that was too small to provide useful feedback during training. Fixing this required going back to the data, pooling and resplitting it properly, and retraining from scratch. The resulting model is significantly more balanced and behaves predictably on both classes.